Desde que uso Pixi estoy tan Pichi

Workspaces ROS 2 declarativos y reproducibles con Pixi y RoboStack

Francisco Martín Rico
fmrico@gmail.com

Esteve Fernández
esteve.fernandez@gmail.com

Juan Sebastián Cely
juan.cely@urjc.es

Francisco Miguel Moreno
franciscom.moreno@urjc.es

Hoy

Duración	Bloque	Formato
10 min	Bienvenida y comprobación de la preparación	juntos
15 min	Robótica para todos	hablamos
25 min	Pixi en 25 minutos	hablamos
45 min	**Ejercicio 1:** tu primer workspace ROS 2	escribís
15 min	Construir paquetes ROS con Pixi	hablamos
35 min	**Ejercicio 2:** construir y publicar paquetes	escribís
10 min	Colaboración, CI/CD y Docker	hablamos
25 min	**Ejercicio 3:** listo para tu equipo	escribís
extra	Ejercicio 4 (opcional): Nav2 con `pixi-ros`	demo

Todo el material: `ejercicios/` (instrucciones paso a paso) y `soluciones/` (workspaces terminados) en el repositorio del taller.

Bienvenida

Que vuestro workspace ROS funcione en cualquier máquina.

Antes de empezar...

- ¿Cuántos habéis tenido un `rosdep install` que falla distinto en vuestro portátil que en el del compañero?
- ¿Cuántos tenéis un `Dockerfile` de ROS 2 de más de 150 líneas solo para fijar versiones?
- ¿Cuántos tenéis instalado más de un `/opt/ros/<distro>` en la misma máquina a la vez?

No sois los únicos. Este taller va de eso.

¿Lo tenéis preparado?

```
curl -fsSL https://pixi.sh/install.sh | sh     # pixi --version >= 0.77
git clone <repositorio-del-taller>
cd workshop-pixi
pixi install --all --manifest-path soluciones/01-workspace-ros/pixi.toml
pixi run --manifest-path soluciones/01-workspace-ros/pixi.toml sim
```

¿Aparece una tortuga? Estáis listos.

¿Tenéis ROS instalado en el sistema?

Comentad el `source /opt/ros/...` de vuestro `~/.bashrc` durante el taller: sus variables se cuelan en el entorno de Pixi y mezclan distros.

Robótica para todos

Por qué la forma de instalar ROS deja gente fuera.

Entrar en ROS va atado a un SO y a una distro

Tienes	**Tu camino**
Ubuntu 24.04	`apt install`, el camino feliz
Ubuntu 22.04	Solo Humble. ¿Quieres Jazzy? Reinstala el SO
Arch u otra Linux	Compilar desde fuente, o un contenedor
macOS	Sin camino oficial. VM o Docker
Windows	Soporte limitado, WSL2 o Docker

Y además

`apt install ros-jazzy-desktop` hoy y dentro de 6 meses instala builds distintos: no hay lockfile. `rosdep` llama a `apt`, `pip`… sin fijar versiones. Y el modelo underlay/overlay (`source ... setup.bash`) es manual y frágil.

"Usa Docker" es la respuesta aceptada

- La instalación se vuelve posible, y el equipo comparte una imagen.
- Pero tu flujo de trabajo vive dentro de una caja: editor, depurador, USB, red, GPU, GUIs... todo cuesta más.
- Docker aísla, pero **no fija el contenido**: un `apt-get install` dentro del Dockerfile no es reproducible si no congelas todo a mano.
- Se monta una vez, reconstruir es lento, y nadie lo vuelve a tocar.

Mensaje clave

Docker sigue siendo válido para desplegar, pero no debería ser la **única** fuente de verdad de las dependencias del proyecto.

Lo que queremos

```
git clone tu-proyecto
cd tu-proyecto
pixi run application
```

Cualquier máquina, tres comandos, y tu proyecto de robótica funciona.

¿El equivalente a un `package-lock.json` o un `Cargo.lock`, **para ROS 2**?

¿Cómo llegamos ahí?

Todas las máquinas · Sin distro · Declarativo · Reproducible · Rápido · Simple

Cómo llegar (1/2)

Todas las máquinas

Linux, macOS, Windows; x86 y arm64. Tu portátil, la CI y el robot pueden ser máquinas distintas.

Sin distro

Una distro de ROS es un proceso de release, no un sistema operativo. Jazzy en Ubuntu 22.04, Kilted en macOS, Humble en Windows… y dos a la vez.

Declarativo

Tu entorno es un fichero en git: `pixi.toml` dice lo que quieres. Vuelves a un estado bueno como vuelves atrás en el código.

Cómo llegar (2/2)

Reproducible

Un lockfile (`pixi.lock`) en lugar de una imagen: cada paquete, versión y hash. El mismo entorno en el portátil, en la CI y en el robot.

Rápido

Minutos hasta tener el proyecto funcionando. Que lo único lento sea la conexión.

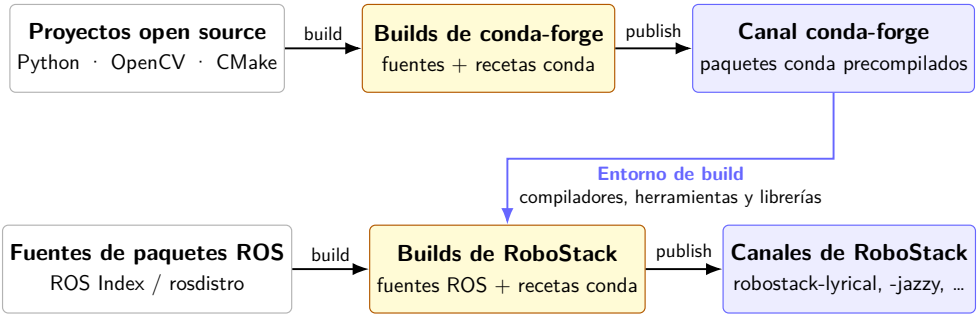
Simple

Si sabes usar `pip`, sabes usar Pixi: `pixi init`, `pixi add`, `pixi run`. Sin `apt`, sin `rosdep`, sin `source`, sin `sudo`.

¿Por qué es tan simple?

No tenéis que construir vosotros todo ROS.

Sobre hombros de gigantes: conda-forge y RoboStack



Pixi combina paquetes de **conda-forge** y de **RoboStack** en un mismo entorno.

Un dato honesto

RoboStack es hoy Tier 3

En la clasificación oficial de plataformas de ROS: soporte **comunitario**, no de Open Robotics.

- Con adopción creciente, y anunciado como parte de la estrategia de accesibilidad de Open Robotics.
- Mantenerlo es trabajo duro: Tobias Fischer, Silvio Traversaro, Daisuke Nishimatsu y muchos más.
- ¿Falta un paquete en RoboStack? Es un pull request a `vinca.yaml`.

Pixi en 25 minutos

Lo básico que necesitáis para el Ejercicio 1.

¿Qué es Pixi?

Gestor de paquetes	Qué instala
`apt`	Paquetes Debian
`brew`	Paquetes Homebrew
`pip`	Paquetes Python de PyPI
`pixi`	**paquetes conda + paquetes Python de PyPI**

- Escrito en Rust sobre **Rattler** (reimplementación del ecosistema conda), por el equipo de **prefix.dev**, autores de **mamba**.
- Sin entorno `base`: un binario estático. Todo vive en la carpeta del proyecto, con `pixi.toml` + `pixi.lock`.

? Con lo que sabéis de `pip` o `npm`, ¿cómo creéis que se instala ROS 2 con Pixi?

Instalar ROS 2

Instalación oficial en Ubuntu	**Con Pixi + RoboStack**
Ubuntu 24.04 y un locale UTF-8	Pixi en Linux, macOS o Windows
Universe + `ros2-apt-source`	`pixi init -c robostack-lyrical -c conda-forge`
`sudo apt update && sudo apt upgrade`	Sin tocar el sistema
`sudo apt install ros-lyrical-desktop`	`pixi add ros-lyrical-desktop`
`source /opt/ros/lyrical/setup.bash`	`pixi shell`
`ros2 run demo_nodes_cpp talker`	`ros2 run demo_nodes_cpp talker`

El `pixi.toml`

```toml
[workspace]
# Para qué máquinas se resuelve (y se genera el lockfile)
platforms = ["linux-64", "osx-arm64", "win-64"]
# De dónde salen los paquetes: RoboStack para ROS, conda-forge para el resto
channels = ["robostack-lyrical", "conda-forge"]

[dependencies]
ros-lyrical-desktop = "*"

[tasks]
start = "ros2 launch my_package my_launch_file.launch.py"
```

Vive en la **raíz** de tu workspace. Además: `[activation]`, `[environments]`, `[target.<plataforma>]`, `[pypi-dependencies]`…

El comando `pixi`

Comando	Qué hace
`pixi init`	Crea un workspace
`pixi add ros-lyrical-ros-base`	Añade la dependencia, resuelve e instala
`pixi run ros2 topic list`	Ejecuta un comando dentro del entorno
`pixi shell`	Abre un shell dentro del entorno
`pixi install`	Instala el entorno desde el lockfile
`pixi list` / `pixi tree`	Qué hay instalado, y por qué
`pixi global install gh`	Herramientas CLI globales, aisladas

`pixi <comando> --help` · chuleta en `referencia/chuleta.md`

Tasks

```toml
[tasks]
sim = "ros2 run turtlesim turtlesim_node"

[tasks.build]
cmd = "colcon build"
inputs = ["src/turtle_dancer"]

[tasks.dance]
cmd = "ros2 run turtle_dancer dance"
depends-on = ["build"]
```

```
# En todas las plataformas
pixi run sim

# Compila antes
pixi run dance

# Cualquier comando
pixi run ros2 topic list
```

- Se ejecutan en un shell multiplataforma propio (`deno_task_shell`), dentro del entorno.
- `depends-on` encadena; `inputs`/`outputs` cachean: no se repite si nada cambió.

pixi shell y la activación

```
pixi shell
ros2 topic list
ros2 run turtlesim turtlesim_node
exit
```

```
[activation]
scripts = ["install/setup.sh"]

[activation.env]
ROS_DOMAIN_ID = "42"
```

- `pixi shell` te mete en el entorno, como hacía `source install/setup.bash`.
- `pixi run` y `pixi shell` activan el entorno primero, incluidos tus scripts.
- `pixi shell-hook` imprime la activación como un script de shell normal (lo usaremos en Docker).

El lockfile: el porqué de la reproducibilidad

- `pixi.lock` fija versión exacta, build y **sha256** de cada paquete (conda y PyPI), **para cada plataforma declarada**.
- Se actualiza solo con `pixi add` / `pixi update`.
- `pixi install --locked` → falla si el lockfile no está sincronizado (la CI).
- `pixi lock --check` → la barrera barata en cada pull request.

Se commitea a git

Todo el equipo, la CI y el robot instalan exactamente los mismos binarios. Es más fácil tirarlo que reconstruirlo cuando algo se actualizó sin que te enterases... justo antes de la demo.

Compilar tu paquete con colcon

Con ROS instalado y `src/turtle_dancer/` en tu workspace:

```
pixi add ros-dev-tools        # colcon, CMake y compiladores
pixi run colcon build
pixi workspace activation scripts add --target unix install/setup.sh
pixi run ros2 run turtle_dancer dance
```

- Igual que siempre, pero dentro del entorno: aparecen `build/`, `install/` y `log/`.
- La activación carga el overlay por ti. Nada de `setuptools` ni empy fijados a mano.

Dos distros, un workspace

```
[environments.default]
channels = ["robostack-lyrical", "conda-forge"]
[environments.default.dependencies]
ros-lyrical-ros-base = ">=0.13"

[environments.jazzy]
channels = ["robostack-jazzy", "conda-forge"]
[environments.jazzy.dependencies]
ros-jazzy-ros-base = "*"
```

Cada entorno tiene sus canales y paquetes; las tasks se comparten. `pixi run sim` para Lyrical, `pixi run -e jazzy sim` para Jazzy.

Imposible o muy incómodo con `apt`. Con Pixi, un entorno más.

Resolver para máquinas que no tienes

```toml
[workspace]
platforms = [
  "linux-64", "osx-arm64", "win-64",
  { name = "cuda-linux-64", platform = "linux-64", cuda = "12" },
  { name = "jetson", platform = "linux-aarch64", cuda = "13" },
]
```

```
pixi lock                          # resuelve todas las plataformas
pixi list --platform jetson        # lo que tendría la Jetson
```

Resolver es barato y portable; instalar es lo caro, y se hace donde toca.

Paquetes virtuales y CUDA

```
$ pixi info
Virtual packages: __unix=0=0
  : __linux=6.8.0=0
  : __glibc=2.39=0
  : __cuda=12.4=0
  : __archspec=1=x86_64
```

```
[dependencies]
# GPU donde hay CUDA, CPU en el resto
pytorch-gpu = { version = ">=2.5", when = "__cuda" }
pytorch = ">=2.5"
```

- No son paquetes que instalas: son hechos sobre la máquina. El solver los usa.
- En una máquina que no tienes, los declaras tú (`cuda = "12"`).
- El driver viene del SO; todo lo demás (toolkit, cuDNN, PyTorch) de conda-forge.

Un workspace nuevo en cuatro líneas

```
pixi init ros-workspace -c robostack-lyrical -c conda-forge
cd ros-workspace
pixi add ros-lyrical-ros-base ros-lyrical-turtlesim
pixi run ros2 run turtlesim turtlesim_node
```

PRÁCTICA

Ejercicio 1: tu primer workspace ROS 2

45 minutos · `cd ejercicios/01-workspace-ros`

Ejercicio 1: tu primer workspace ROS 2

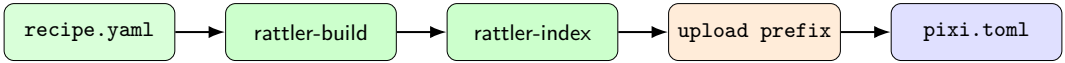
1. `pixi init`, canal `robostack-lyrical`, `ros-lyrical-ros-base` + `turtlesim`.
2. Tasks `sim`, `teleop`, `topics`.
3. Compilar `turtle_dancer` con colcon y cargar el overlay con `[activation]`.
4. Lyrical y Jazzy en el mismo workspace.
5. Un cerebro con PyTorch (`turtle_brain`).
6. Plataformas: CPU, GPU (`cuda-linux-64`) y Jetson, resueltas desde tu portátil.
7. (Opcional) Ejecutarlo en una GPU de verdad.

Instrucciones: `ejercicios/01-workspace-ros/README.md` · ¿Atascado? `soluciones/01-workspace-ros/` o levanta la mano.

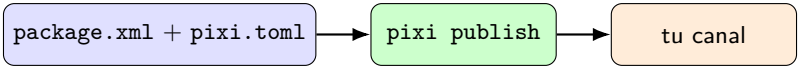
Construir paquetes ROS con Pixi

Sin colcon, sin overlay, sin `source`. Y sin montarse una buildfarm.

Antes: tu propia buildfarm



- Una receta a mano por paquete, duplicando lo que ya dice el `package.xml`.
- Para ROS completo: fork de RoboStack, `vinca`, CI propia…



Ahora: dos líneas junto al `package.xml`, y un comando.

Dos papeles para un `pixi.toml`

Workspace	`[workspace]`	lo que ejecutas con `pixi run`
Paquete	`[package]`	lo que Pixi construye

Todavía en *preview*: puede cambiar algo de sintaxis.

```
pixi workspace preview add pixi-build
pixi workspace dependencies add "pixi-build-ros >=0.7.5"
```

Lo que escribes

`pixi.toml` (workspace)

```toml
[workspace]
preview = ["pixi-build"]

[workspace.dependencies]
pixi-build-ros = ">=0.7.5"

[dependencies]
ros-lyrical-turtle-dancer = { path = "src/turtle_dancer" }
```

`src/turtle_dancer/pixi.toml` (paquete)

```toml
[package.build.backend]
name = "pixi-build-ros"
workspace = true
```

- Nombre de la dependencia: el del `package.xml` con el prefijo de la distro y guiones.
- `CMakeLists.txt` y `setup.py` no se tocan: el backend ejecuta tu build `ament`.

pixi-build-ros lee tu package.xml

Lo que ya tienes

```xml
<package format="3">
  <name>turtle_dancer</name>
  <version>0.1.0</version>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>
  <depend>geometry_msgs</depend>
  <exec_depend>turtlesim</exec_depend>
  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Lo que **no** tienes que escribir

```toml
[package]
name = "ros-lyrical-turtle-dancer"
version = "0.1.0"
[package.build-dependencies]
ros-lyrical-ament-cmake = "*"
ros-lyrical-rclcpp = "*"
ros-lyrical-geometry-msgs = "*"
[package.run-dependencies]
ros-lyrical-rclcpp = "*"
ros-lyrical-geometry-msgs = "*"
ros-lyrical-turtlesim = "*"
```

El `package.xml` sigue siendo la única fuente de verdad. La distro la elige el canal del workspace.

El bucle editar-ejecutar

- `pixi install` construye el paquete en un entorno de build aislado y lo instala en tu entorno **como un paquete conda**, al lado de `turtlesim`.
- `pixi run dance` comprueba antes las entradas (fuentes, `package.xml`, `CMakeLists.txt`, launch, mensajes…) y reconstruye si algo cambió.
- Fuera: la task `build`, `ros-dev-tools`, la activación, `build/ install/ log/`.

Editar → `pixi run` → funciona. Desde una terminal nueva, sin `source` de nada.

Backends de build

Pixi no sabe compilar tu código. Un **backend** sí.

Backend	Construye	Lee
`pixi-build-ros`	Paquetes ROS	`package.xml`
`pixi-build-cmake`	C y C++	`CMakeLists.txt`
`pixi-build-python`	Python	`pyproject.toml`
`pixi-build-rust`	Rust	`Cargo.toml`
`pixi-build-rattler-build`	Recetas conda	`recipe.yaml`

¿Necesitas control total (parches, pasos de build propios)? Tu `recipe.yaml` de `rattler-build` sigue valiendo: `[package.build.config] recipe = "recipe.yaml"`.

Publicar

Cada paquete decide si se publica:

```toml
# src/turtle_dancer/pixi.toml
[package]
publish = true
```

```
# Canal local, como en el Ejercicio 2
pixi publish --target-channel output

# Tu canal de prefix.dev (cuenta gratuita, permiso de escritura)
pixi auth login prefix.dev
pixi publish --target-channel https://prefix.dev/<tu-canal>
```

Construye en orden de dependencias todos los paquetes con `publish = true` y crea el índice del canal. Para otras plataformas: `--target-platform`, o una matriz en la CI.

Depender de un paquete

```toml
[workspace]
channels = [
  "https://prefix.dev/<tu-canal>",
  "https://prefix.dev/robostack-lyrical",
  "conda-forge",
]

[dependencies]
# Ya construido, desde un canal
ros-lyrical-turtle-dancer = "*"
```

```toml
# Desde el código, local
[dependencies.ros-lyrical-turtle-dancer]
path = "src/turtle_dancer"

# Desde git
[dependencies.brain]
git = "https://github.com/user/brain.git"
branch = "main"
```

El orden de los canales importa: tu canal → RoboStack → conda-forge. El mismo patrón que la buildfarm, sin buildfarm.

PRÁCTICA

Ejercicio 2: construir y publicar paquetes ROS

35 minutos · `cd ejercicios/02-paquetes-ros`

Ejercicio 2: construir y publicar paquetes ROS

1. Activar `pixi-build` y añadir el backend al workspace.
2. Escribir `src/turtle_dancer/pixi.toml`: dos líneas.
3. Añadir la dependencia `path`, `pixi install`.
4. Borrar lo de colcon, `pixi run dance`.
5. Cambiar `dance.cpp`, volver a ejecutar.
6. Lo mismo con `src/turtle_choreographer` (Python).
7. Publicar los dos paquetes en un canal (local o prefix.dev).
8. Instalarlos desde otro workspace, sin código fuente.

Instrucciones: `ejercicios/02-paquetes-ros/README.md`

Colaboración, CI/CD y Docker

De «funciona en mi máquina» a «funciona en la del equipo».

CI en dos líneas

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: prefix-dev/setup-pixi@v0.10.2
        with:
          locked: true
      - run: pixi run test
```

- Instala Pixi y el entorno desde el lockfile; ejecuta tu task.
- Las mismas tasks que en tu portátil: no hay un segundo setup en YAML que se desincronice.
- Caché gratis, con el hash de `pixi.lock` como clave.

Docker: la etapa de build

```
FROM ghcr.io/prefix-dev/pixi:0.80.0-noble AS build
WORKDIR /app
COPY pixi.toml pixi.lock ./
RUN pixi install --locked
RUN pixi shell-hook -s bash > /shell-hook.sh \
    && echo 'exec "$@"' >> /shell-hook.sh
```

- Parte de la imagen de Pixi y copia el manifiesto y el lockfile.
- `--locked`: falla si el lockfile no cuadra; la imagen no puede contener algo que el equipo no probó.
- Escribe la activación en un script, para la siguiente etapa.

Docker: la etapa final

```
FROM ubuntu:26.04 AS runtime
COPY --from=build /app/.pixi/envs/default /app/.pixi/envs/default
COPY --from=build /shell-hook.sh /shell-hook.sh
ENTRYPOINT ["/bin/bash", "/shell-hook.sh"]
CMD ["ros2", "run", "demo_nodes_cpp", "talker"]
```

- Una imagen base normal, sin el binario de `pixi`.
- Recibe el entorno terminado y el script de activación.
- Portátil, CI e imagen instalan desde el mismo `pixi.lock`.

pixi-pack: un entorno en un fichero

En tu portátil

```
pixi-pack --platform linux-
    aarch64 \
  pixi.toml
```

En el robot

```
pixi-unpack environment.tar
source activate.sh
ros2 run demo_nodes_cpp talker
```

- Para despliegues sin red, con 4G poco fiable o sin Docker.
- En el destino no hace falta Pixi, ni conda, ni conexión.
- `--create-executable`: un único fichero autoextraíble.

PRÁCTICA

Ejercicio 3: listo para tu equipo

25 minutos · `cd ejercicios/03-colaboracion`

Ejercicio 3: listo para tu equipo

1. `pixi global install gh` y subir el workspace a GitHub.
2. CI con `setup-pixi`: verla en verde.
3. Completar el `Dockerfile`, construir la imagen y ejecutarla: sin Pixi dentro.
4. Empaquetar el entorno, desempaquetarlo y ejecutar sin Pixi.

Instrucciones: `ejercicios/03-colaboracion/README.md`

Un workspace real

Ejercicio 4 (opcional): Nav2 con `pixi-ros`, sin traducir dependencias a mano.

Ejercicio 4 (opcional): un workspace real

- Hasta ahora, paquetes de juguete. ¿Y un workspace grande que hoy compiláis con `rosdep` y `colcon`?
- `rosdep` no sirve dentro de Pixi: llama a `apt`/`pip` del sistema.

`pixi-ros`

Escanea los `package.xml` del workspace y genera el `pixi.toml` con las dependencias ya traducidas a paquetes de RoboStack y conda-forge, y tasks `build`/`test` con colcon. El `rosdep install` moderno, pero reproducible.

Aviso de timing

Nav2 son más de 80 paquetes C++. El build se lanza con antelación y aquí enseñamos el resultado en vivo.

Nav2 en siete comandos

```
pixi global install pixi-ros
mkdir -p nav2_ws/src && cd nav2_ws
git clone -b jazzy https://github.com/ros-navigation/navigation2.git src/
    navigation2
pixi-ros init --distro jazzy --platform linux-64
pixi add gcc_linux-64=13.3.0 gxx_linux-64=13.3.0 libboost=1.88.0 libboost-devel
    =1.88.0 ros-jazzy-rviz2
pixi run build
pixi run ros2 launch nav2_bringup tb3_simulation_launch.py
```

- GCC y Boost se fijan a las versiones de Ubuntu 24.04 con las que compila Nav2 Jazzy; RViz2 no es dependencia de build.
- En RViz2: **2D Pose Estimate** → **Nav2 Goal** → el robot navega solo. Gazebo, RViz2 y Nav2 en un entorno Pixi, sin `sudo`, sin `apt`, sin Docker.

Con vuestro propio workspace

1. `pixi-ros init --distro <distro>` en la raíz: revisad los `NOT FOUND`.
2. `pixi run build` con colcon, como en el Ejercicio 1.
3. Paquete a paquete (empezando por las hojas), un `pixi.toml` de dos líneas y una dependencia `path`, como en el Ejercicio 2.
4. `publish = true` y `pixi publish` desde la CI: vuestra "buildfarm".
5. `setup-pixi` en la CI y `pixi-pack` o Docker para el robot, como en el Ejercicio 3.

Si una dependencia del `package.xml` no está en RoboStack con ese nombre: `extra-package-mappings` en la configuración de `pixi-build-ros`.

Cierre

Lo que os lleváis, y dónde seguir.

Lo que os lleváis

- Un workspace ROS 2 en un `pixi.toml` + `pixi.lock`: `git clone`, `cd`, `pixi run`.
- Varias distros, varias plataformas y GPU/Jetson resueltas desde vuestro portátil.
- Paquetes ROS construidos desde su `package.xml` y publicados con `pixi publish`.
- La misma definición en el portátil, en la CI, en Docker y en el robot.

Recursos

- Pixi: `https://pixi.prefix.dev` — ROS: `https://pixi.prefix.dev/latest/build/ros/`
- RoboStack: `https://robostack.github.io/` — `https://github.com/RoboStack`
- pixi-ros: `https://github.com/ruben-arts/pixi-ros` — Nav2: `https://github.com/ros-navigation/navigation2`
- Discord de Pixi, canal `#pixi`: `https://discord.gg/kKV8ZxyzY4`
- Paper original: Fischer et al., *"A RoboStack Tutorial"*, IEEE RAM 2021 (arXiv:2104.12910)

Agradecimientos

Basado en el taller de prefix.dev en ROSCon 2026

Declarative ROS workspaces with Pixi and RoboStack, de Ruben Arts, Wolf Vollprecht y Bas Zalmstra (BSD-3-Clause).

`https://prefix-dev.github.io/roscon-2026-declarative-ros-workspaces-with-pixi/`

Los paquetes `turtle_dancer`, `turtle_choreographer` y `turtle_brain` de los ejercicios vienen de ese taller.

Y a la comunidad de conda-forge y RoboStack, sin la que nada de esto funcionaría.

Preguntas